

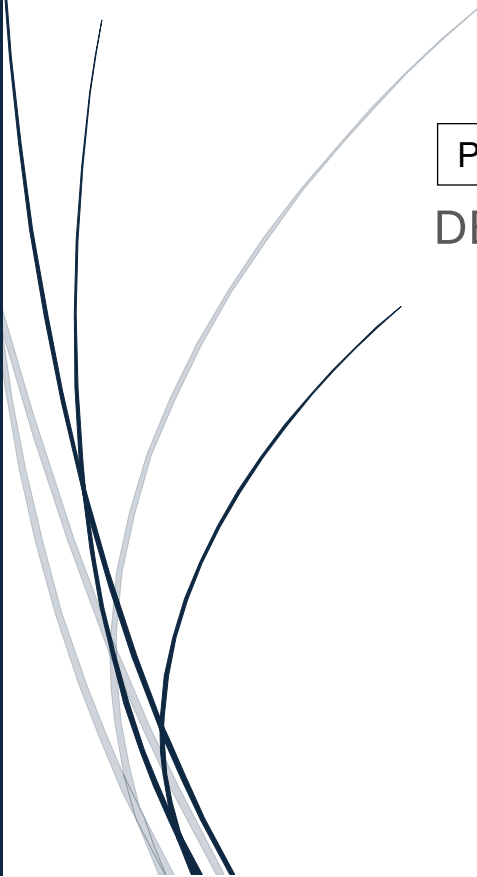


5/2/2026

Imran Nazeer

DHC-2094 (Cybersecurity Intern)

<https://github.com/ink-baltee/DevHub-Cybersecurity-Internship-2026>



Phase 1 Week 1 Report

DEVELOPERSHUB CORPORATION

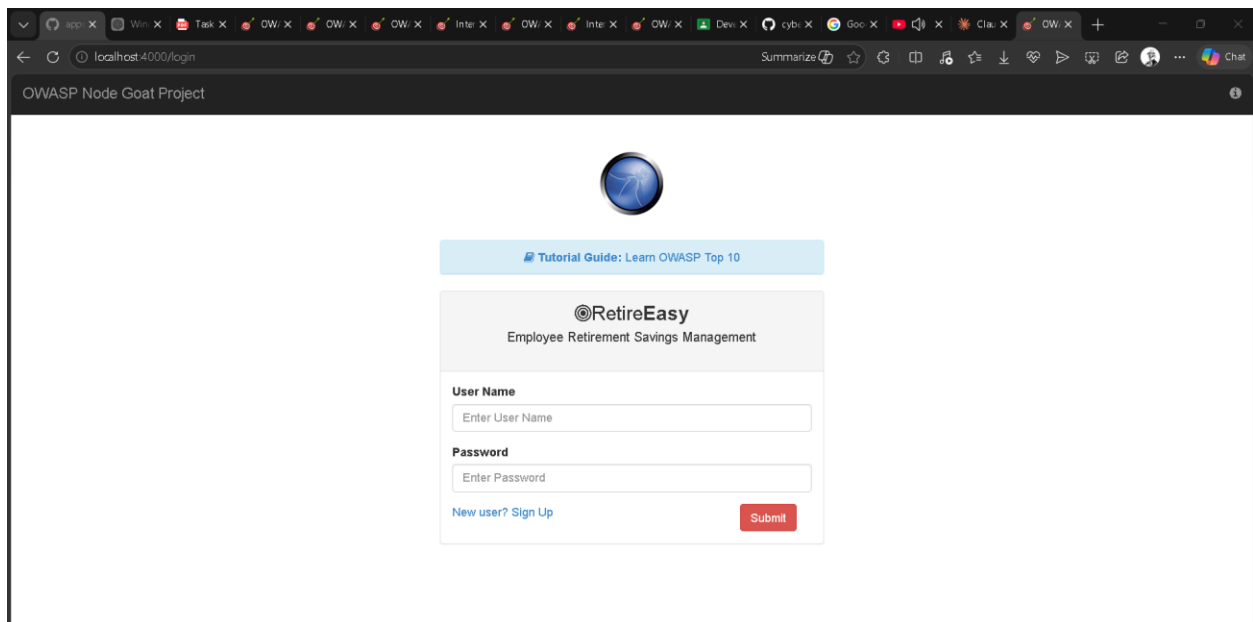
Contents

1. Overview	4
1.1 Objectives.....	4
1.2 Scope of Assessment.....	4
1.3 Summary of Findings.....	4
2. Setup Process.....	6
2.1 Tools & Software Installed	6
2.2 Issues Encountered During Setup	6
2.3 Step-by-Step Setup Log	6
Step 1: Install Node.js	6
Step 2: Install Git.....	7
Step 3: Install MongoDB 4.4.29.....	7
Step 4: Clone and Install NodeGoat.....	8
Step 5: Seed Database & Start App.....	8
Step 6: Install Java JDK 21 & OWASP ZAP	8
3. Application Details.....	10
3.1 General Information	10
3.2 Application Pages.....	10
4. Vulnerability Assessment	11
4.1 Testing Methodology	11
4.2 Manual Testing, Vulnerability Summary.....	11
4.3 OWASP ZAP Automated Scan	11
4.3.1 ZAP Scan, Alert Count Summary	13
4.3.2 ZAP Scan, High Severity Findings	13
4.3.3 ZAP Scan, Medium Severity Findings	14
4.3.4 ZAP Scan, Low Severity Findings	14
4.3.5 ZAP Scan, Informational Findings	15
5. Detailed Manual Testing Findings	16
Finding 1, NoSQL Injection.....	16
5.1.1 Description.....	16
5.1.2 Test Steps & Evidence	16
5.1.3 Impact.....	16
5.1.4 Recommendations	17
Finding 2, Injection / Code Parsing Error	17
5.2.1 Description.....	17
5.2.2 Test Steps & Evidence	17

5.2.3 Impact	17
5.2.4 Recommendations	18
Finding 3, Open Redirect / JavaScript URI Injection	18
5.3.1 Description	18
5.3.2 Test Steps & Evidence	18
5.3.3 Impact	19
5.3.4 Recommendations	19
Finding 4, Weak Password Policy	19
5.4.1 Description	20
5.4.2 Test Steps & Evidence	20
5.4.3 Impact	21
5.4.4 Recommendations	21
Finding 5, Verbose Error Message Disclosure	21
5.5.1 Description	21
5.5.2 Evidence	22
5.5.3 Recommendations	22
Finding 6, Incorrect Input Validation on Profile Page	22
5.6.1 Description	22
5.6.2 Evidence	23
5.6.3 Recommendations	23
6. Risk Matrix	24
7. Areas of Improvement	25
7.1 Input Validation & Sanitization	25
7.2 Password Security	25
7.3 Authentication & Session Management	25
7.4 Security Headers (via Helmet.js)	25
7.5 Error Handling & Logging	25
7.6 General Security Hygiene	25
8. Week 2 Remediation Plan	26
9. Conclusion	27

Target Application:
OWASP NodeGoat v1.3.0 RetireEasy

Field	Details
Prepared By	Imran Nazeer (DevHub Cybersecurity Intern)
Internship Phase	Phase 1, Week 1
Assessment Date	2 May 2026
Report Classification	Internal / Training Use Only
Target URL	http://localhost:4000
Assessment Type	Manual + Automated (OWASP ZAP) Security Assessment
Total Vulnerabilities Found	27 (ZAP) + 6 Manual = 33 total findings



Screenshot 1: NodeGoat Application Home Page

Capture http://localhost:4000 showing the RetireEasy login page in your browser

1. Overview

This report presents the complete findings of the Week 1 Security Assessment conducted as part of the DevelopersHub Cybersecurity Internship, Phase 1. The objective of this week was to understand a vulnerable web application, perform both manual and automated security testing, and document all findings in a structured professional report.

The target application “OWASP NodeGoat” is an intentionally insecure Node.js web application developed and maintained by the OWASP Foundation for cybersecurity education. It simulates a real-world retirement savings management portal called 'RetireEasy' and contains multiple deliberately introduced vulnerabilities representing the OWASP Top 10.

Testing combined manual black-box testing techniques with an automated OWASP ZAP scan, resulting in a total of 33 unique findings across Critical, High, Medium, Low, and Informational severity levels.

1.1 Objectives

1. Download, configure, and successfully run the NodeGoat vulnerable web application
2. Perform manual vulnerability testing across key pages including login, signup, profile, allocations, contributions, and memos
3. Run an automated OWASP ZAP scan to identify additional vulnerabilities
4. Document all discovered vulnerabilities with evidence, impact analysis, and recommendations
5. Produce a structured security assessment report for submission

1.2 Scope of Assessment

Scope Item	Details
In Scope, Application	OWASP NodeGoat v1.3.0
In Scope, URL	http://localhost:4000 (local environment only)
In Scope, Pages	/login, /signup, /profile, /allocations, /contributions, /memos, /learn, /tutorial
In Scope, Test Types	XSS, Injection, Auth Bypass, Password Policy, Redirect, Header Analysis, ZAP Automated Scan
Out of Scope	External URLs, production systems, third-party APIs
Testing Window	Week 1 of DevHub Internship Phase 1 (May 2026)

1.3 Summary of Findings

Severity	Manual Testing	OWASP ZAP Scan	Total
Critical	2	0	2
High	2	7	9
Medium	1	5	6

Low	1	7	8
Informational	0	8	8
Total	6	27	33

2. Setup Process

This section documents the complete environment setup process, including every tool installed, configuration change made, and issue encountered before testing began.

2.1 Tools & Software Installed

Tool	Version	Purpose
Node.js	v24.15.0	JavaScript runtime environment for NodeGoat
npm	v11.x	Package manager for installing dependencies
Git	Latest	Cloning the NodeGoat repository from GitHub
MongoDB	4.4.29	Database backend required by NodeGoat
Java JDK	21	Required runtime for OWASP ZAP
OWASP NodeGoat	v1.3.0	Target vulnerable web application
OWASP ZAP	Latest stable	Automated web application vulnerability scanner
Web Browser	Chrome/Firefox	Manual testing and request inspection

2.2 Issues Encountered During Setup

Issue	Cause	Resolution
'git' not recognized	Git not installed	Downloaded and installed Git from git-scm.com
npm install failed on dvna	Node v24 incompatible with libxmljs native module	Switched to NodeGoat (no native dependencies)
MongoDB 8.x connection refused	NodeGoat uses legacy OP_QUERY opcodes removed in MongoDB 5.1+	Uninstalled 8.x, installed MongoDB 4.4.29
'mongod' not recognized	MongoDB bin not in system PATH	Added C:\Program Files\MongoDB\Server\4.4\bin to PATH
'net start MongoDB', Access Denied	PowerShell not running as Administrator	Reopened PowerShell with Run as Administrator
App crash on signup	Database not seeded with initial data	Ran: npm run db:seed
ZAP won't launch	Java not installed	Installed Java JDK 21 from oracle.com

2.3 Step-by-Step Setup Log

Step 1: Install Node.js

6. Visited <https://nodejs.org> and downloaded the LTS installer for Windows

7. Ran the .msi installer with all default settings
8. Verified: node -v → v24.15.0, npm -v → v11.x

```
PS C:\WINDOWS\system32> node -v
v24.15.0
PS C:\WINDOWS\system32> npm -v
11.12.1
PS C:\WINDOWS\system32> _
```

Screenshot 2: Node.js & npm Version Confirmation

PowerShell showing 'node -v' and 'npm -v' output confirming successful installation

Step 2: Install Git

9. Visited <https://git-scm.com/download/win> and downloaded installer
10. Ran with all default options, reopened PowerShell
11. Verified: git --version

Step 3: Install MongoDB 4.4.29

12. Visited <https://www.mongodb.com/try/download/community>
13. Selected: Version 4.4.29, Platform: Windows, Package: MSI
14. Installed with 'Install MongoDB as a Service' checked
15. Added C:\Program Files\MongoDB\Server\4.4\bin to system PATH
16. Started service as Administrator: net start MongoDB
17. Verified: mongod --version → db version v4.4.29

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> mongod --version
db version v4.4.29
Build Info: {
  "version": "4.4.29",
  "gitVersion": "f4dda329a99811c707eb06d05ad023599f9be263",
  "modules": [],
  "allocator": "tcmalloc",
  "environment": {
    "distmod": "windows",
    "distarch": "x86_64",
    "target_arch": "x86_64"
  }
}
PS C:\WINDOWS\system32> _
```

Screenshot 3: MongoDB 4.4.29 Version Confirmation

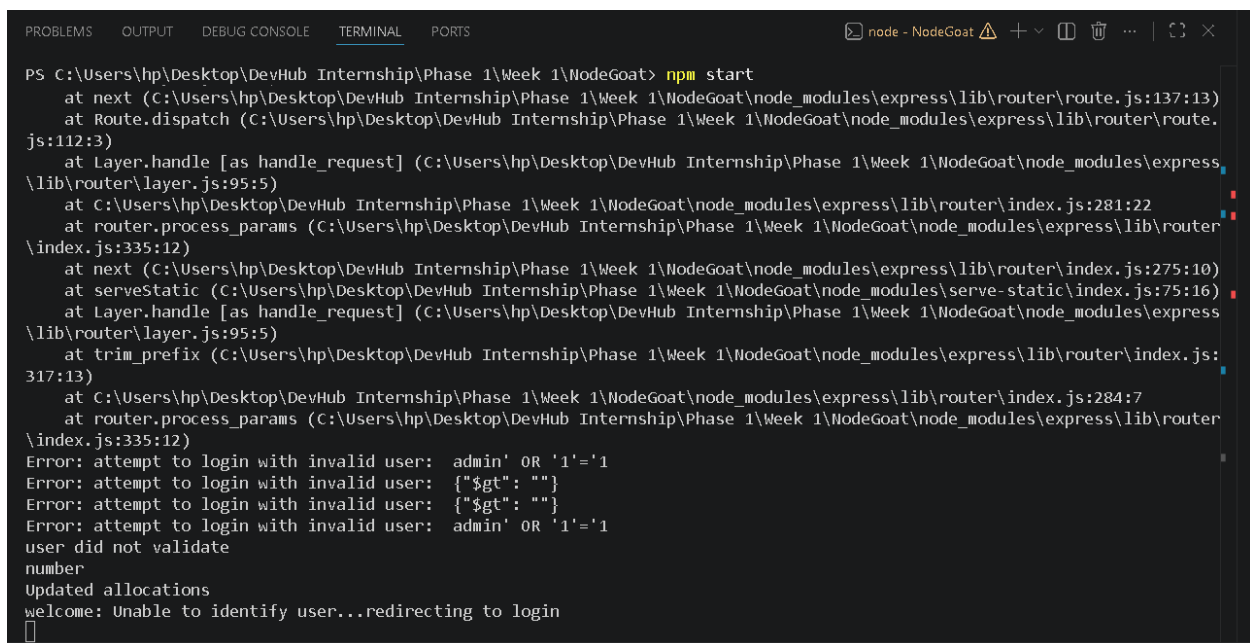
PowerShell showing 'mongod --version' output confirming db version v4.4.29

Step 4: Clone and Install NodeGoat

18. Cloned: `git clone https://github.com/OWASP/NodeGoat.git`
19. Entered folder: `cd NodeGoat`
20. Installed dependencies: `npm install` (962 packages, ~20 minutes)
21. 141 vulnerabilities reported, expected for an intentionally vulnerable app

Step 5: Seed Database & Start App

22. Seeded database: `npm run db:seed`
23. Started app: `npm start`
24. Confirmed: Connected to the database + Express http server listening on port 4000
25. Opened `http://localhost:4000`, RetireEasy login page loaded successfully



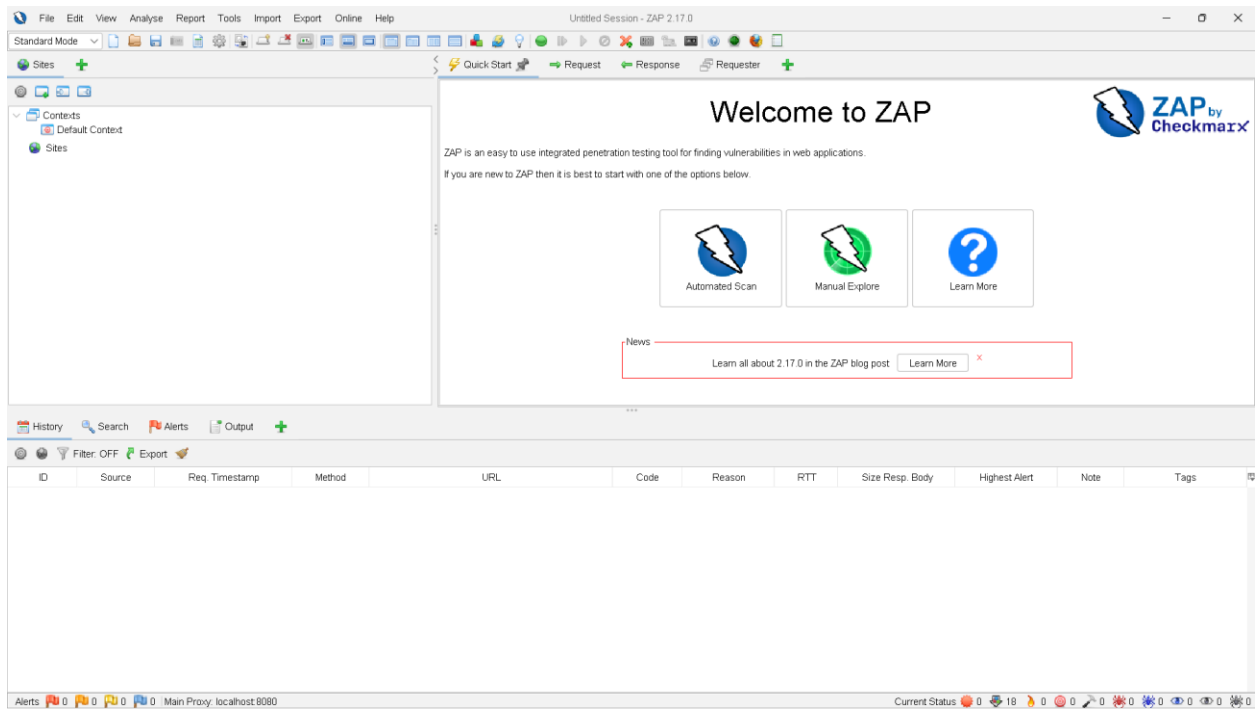
```
PS C:\Users\hp\Desktop\DevHub Internship\Phase 1\Week 1\NodeGoat> npm start
at next (C:\Users\hp\Desktop\DevHub Internship\Phase 1\Week 1\NodeGoat\node_modules\express\lib\router\route.js:137:13)
at Route.dispatch (C:\Users\hp\Desktop\DevHub Internship\Phase 1\Week 1\NodeGoat\node_modules\express\lib\router\route.js:112:3)
at Layer.handle [as handle_request] (C:\Users\hp\Desktop\DevHub Internship\Phase 1\Week 1\NodeGoat\node_modules\express\lib\router\layer.js:95:5)
at C:\Users\hp\Desktop\DevHub Internship\Phase 1\Week 1\NodeGoat\node_modules\express\lib\router\index.js:281:22
at router.process_params (C:\Users\hp\Desktop\DevHub Internship\Phase 1\Week 1\NodeGoat\node_modules\express\lib\router\index.js:335:12)
at next (C:\Users\hp\Desktop\DevHub Internship\Phase 1\Week 1\NodeGoat\node_modules\express\lib\router\index.js:275:10)
at serveStatic (C:\Users\hp\Desktop\DevHub Internship\Phase 1\Week 1\NodeGoat\node_modules\serve-static\index.js:75:16)
at Layer.handle [as handle_request] (C:\Users\hp\Desktop\DevHub Internship\Phase 1\Week 1\NodeGoat\node_modules\express\lib\router\layer.js:95:5)
at trim_prefix (C:\Users\hp\Desktop\DevHub Internship\Phase 1\Week 1\NodeGoat\node_modules\express\lib\router\index.js:317:13)
at C:\Users\hp\Desktop\DevHub Internship\Phase 1\Week 1\NodeGoat\node_modules\express\lib\router\index.js:284:7
at router.process_params (C:\Users\hp\Desktop\DevHub Internship\Phase 1\Week 1\NodeGoat\node_modules\express\lib\router\index.js:335:12)
Error: attempt to login with invalid user: admin' OR '1'='1
Error: attempt to login with invalid user: {"$gt": ""}
Error: attempt to login with invalid user: {"$gt": ""}
Error: attempt to login with invalid user: admin' OR '1'='1
user did not validate
number
Updated allocations
welcome: Unable to identify user...redirecting to login
█
```

Screenshot 4: npm start Success & App Running

PowerShell showing 'Connected to the database' and browser showing the NodeGoat login page

Step 6: Install Java JDK 21 & OWASP ZAP

26. Visited <https://www.oracle.com/java/technologies/downloads/> and downloaded Java JDK 21
27. Installed Java with default settings
28. Verified: `java -version` → java version '21.x.x'
29. Downloaded OWASP ZAP installer from <https://www.zaproxy.org/download/>
30. Installed ZAP with default settings



Screenshot 5: OWASP ZAP Installed & Open
OWASP ZAP welcome screen open and ready to use

3. Application Details

3.1 General Information

Field	Details
Application Name	OWASP NodeGoat (branded as 'RetireEasy')
Version	1.3.0
Maintained By	OWASP Foundation
Purpose	Intentionally vulnerable Node.js app for OWASP Top 10 training
Repository	https://github.com/OWASP/NodeGoat
Local URL	http://localhost:4000
Default Port	4000
Server Language	JavaScript (Node.js)
Web Framework	Express.js
Database	MongoDB 4.4.29
Template Engine	Swig (deprecated)
Authentication	Session-based using cookies (no JWT)
Password Hashing	bcrypt-nodejs (deprecated package)
Security Headers	None (no Helmet.js)
Input Validation	None detected
Logging	None configured

3.2 Application Pages

Page	URL	Description	Auth Required
Login	/login	User authentication	No
Register	/signup	New user registration	No
Dashboard	/dashboard	Main landing after login	Yes
Profile	/profile	View and edit profile	Yes
Allocations	/allocations/:id	Fund allocation management	Yes
Contributions	/contributions	Contribution tracking	Yes
Memos	/memos	Personal notes	Yes
Learn	/learn	External resource loader	Yes
Tutorial	/tutorial	OWASP Top 10 guide	No

4. Vulnerability Assessment

4.1 Testing Methodology

The assessment combined two approaches: manual black-box testing simulating an external attacker, and automated scanning using OWASP ZAP. Manual testing focused on key application pages using crafted payloads. ZAP performed a full spider and active attack scan against the entire application.

Test Type	Technique	Pages / Scope
XSS, Reflected	Injecting <script> and tags	/profile, /memos, /login
XSS, Persistent	Stored payload retrieval	/memos, /signup
NoSQL Injection	HTML and operator payloads in query inputs	/allocations, /contributions
Auth Bypass	SQL-style operators in login fields	/login
Weak Password	Registering with minimal passwords (e.g. '123')	/signup
Open Redirect	Manipulating the returnUrl URL parameter	/login?returnTo=
Error Disclosure	Observing raw error messages in responses	All pages
Automated Scan	OWASP ZAP Spider + Active Attack	Full application

4.2 Manual Testing, Vulnerability Summary

#	Vulnerability	Location	OWASP Top 10	Severity	Confirmed
1	NoSQL Injection	/allocations/1	A03, Injection	Critical	Yes
2	Injection / Code Parsing	/contributions	A03, Injection	High	Yes
3	Open Redirect / XSS Vector	/login?returnTo=	A01, Broken Access Control	High	Yes
4	Weak Password Policy	/signup	A07, Auth Failures	Critical	Yes
5	Verbose Error Disclosure	Multiple pages	A05, Misconfiguration	Medium	Yes
6	Incorrect Input Validation	/profile	A03, Injection	Low	Yes

4.3 OWASP ZAP Automated Scan

An automated scan was performed using OWASP ZAP against the running NodeGoat application at <http://localhost:4000>. ZAP executed a full Spider crawl followed by an Active Attack scan using the Default Attack Policy.

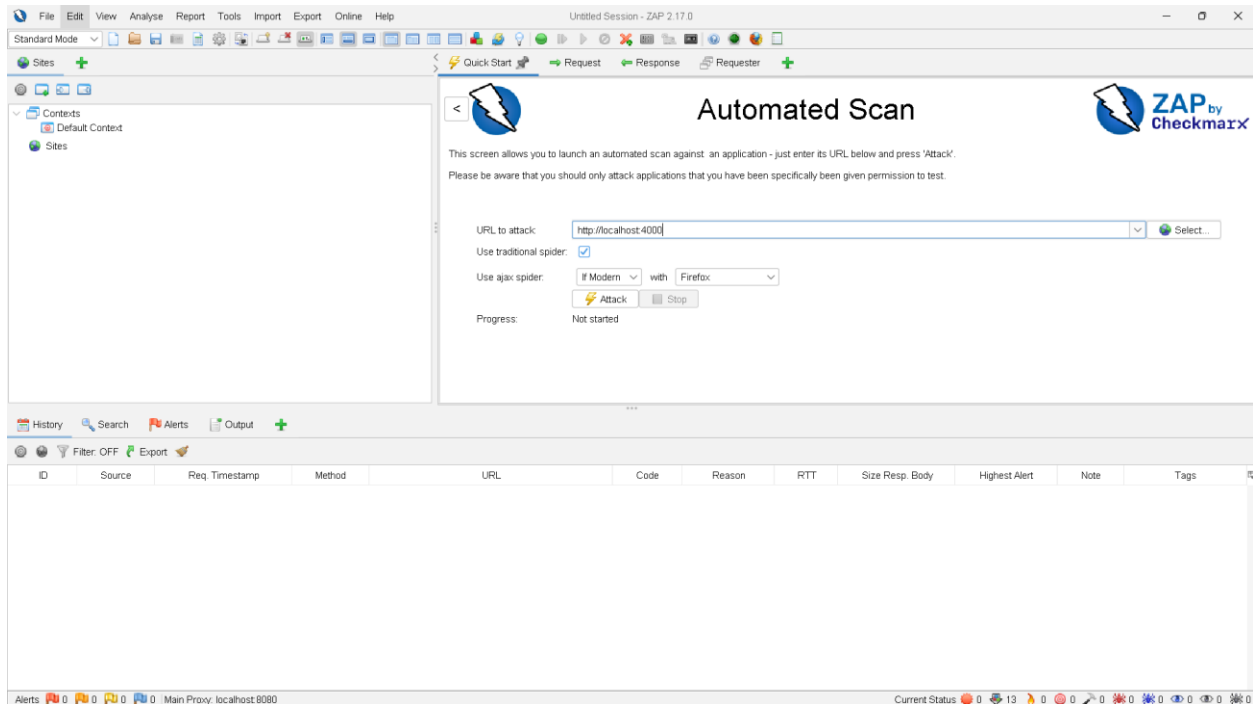
ZAP Scan Type: Automated Scan (Spider + Active Attack)

Target URL: http://localhost:4000

Authentication: Unauthenticated (anonymous user scan)

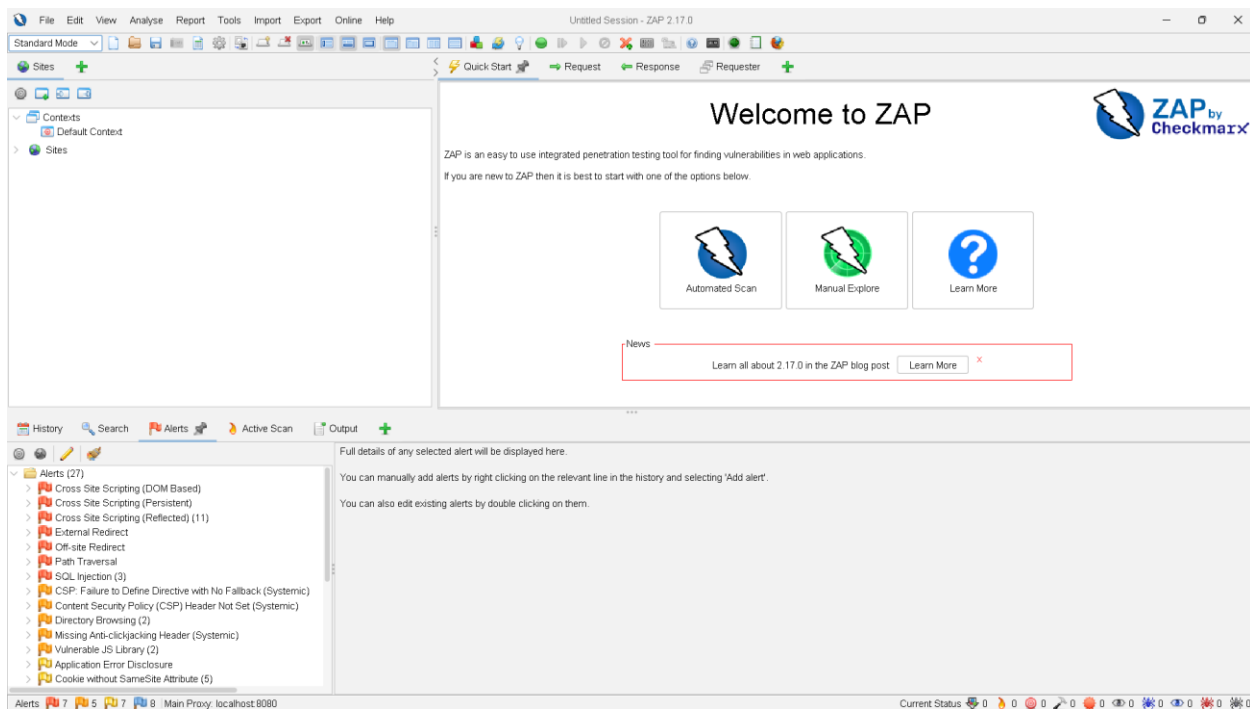
Scan Policy: Default Attack Policy

Total Alerts Found: 27 alerts across 4 risk levels



Screenshot 6: OWASP ZAP Automated Scan Configuration

OWASP ZAP Automated Scan window showing *http://localhost:4000* entered as the target URL before clicking Attack



Screenshot 7: OWASP ZAP Alerts Panel, All 27 Findings

OWASP ZAP Alerts tab showing all 27 discovered vulnerabilities color-coded by severity

4.3.1 ZAP Scan, Alert Count Summary

Risk Level	Count	% of Total	Key Finding Types
High	7	25.9%	XSS (DOM, Persistent, Reflected), SQL Injection, Path Traversal, External Redirect
Medium	5	18.5%	Missing CSP Header, Missing Anti-Clickjacking Header, Directory Browsing, Vulnerable JS Library
Low	7	25.9%	Cookie Issues, Missing Security Headers, Application Error Disclosure, Server Info Leakage
Informational	8	29.6%	Session Management, Authentication Requests, Suspicious Comments, User Agent Fuzzing
Total	27	100%	Full application attack surface covered

4.3.2 ZAP Scan, High Severity Findings

Alert Type	Risk	CWE	Affected URL	OWASP Category
Cross Site Scripting (DOM Based)	High	CWE-79	GET /login#jaVaScRipt:/*...*/oNcliCk=alert()	A03, Injection

Cross Site Scripting (Persistent)	High	CWE-79	GET /memos	A03, Injection
Cross Site Scripting (Reflected)	High	CWE-79	POST /signup	A03, Injection
External Redirect	High	CWE-601	GET /learn?url=https://...owasp.org	A01, Broken Access Control
Off-site Redirect	High	CWE-601	GET /learn?url=https://khanacademy.org	A01, Broken Access Control
Path Traversal	High	CWE-22	GET /allocations/6?threshold=6	A01, Broken Access Control
SQL Injection	High	CWE-89	POST /contributions	A03, Injection

4.3.3 ZAP Scan, Medium Severity Findings

Alert Type	Risk	CWE	Affected URL	Notes
CSP: Failure to Define Directive	Medium	CWE-693	GET /robots.txt	Missing script-src or default-src fallback
Content Security Policy Header Not Set	Medium	CWE-693	GET /	No CSP header on any page, XSS easily exploitable
Directory Browsing Enabled	Medium	CWE-548	GET /tutorial/a1/	Server allows listing of directory contents
Missing Anti-Clickjacking Header	Medium	CWE-1021	GET /	No X-Frame-Options header, clickjacking possible
Vulnerable JS Library	Medium	CWE-1395	GET /vendor/bootstrap/bootstrap.js	Outdated Bootstrap with known vulnerabilities

4.3.4 ZAP Scan, Low Severity Findings

Alert Type	Risk	CWE	Affected URL
Application Error Disclosure	Low	CWE-550	GET /allocations/6?threshold=ZAP
Cookie without SameSite Attribute	Low	CWE-1275	GET /
Cross-Domain JS Source File Inclusion	Low	CWE-829	GET /
Server Leaks Info via X-Powered-By Header	Low	CWE-497	GET /

Strict-Transport-Security Header Not Set	Low	CWE-319	GET /
Timestamp Disclosure, Unix	Low	CWE-497	GET /
X-Content-Type-Options Header Missing	Low	CWE-693	GET /vendor/theme/sb-admin.css

4.3.5 ZAP Scan, Informational Findings

Alert Type	Notes
Authentication Request Identified	ZAP identified POST /login as the authentication endpoint
GET for POST	Login page responds to GET requests, potential issue
Information Disclosure, Suspicious Comments	Sensitive comments found in jquery.min.js source
Modern Web Application	ZAP identified the app as a modern single-page style application
Retrieved from Cache	Some responses served from browser cache
Session Management Response Identified	Session cookie identified on /sitemap.xml
User Agent Fuzzer	App responds differently to various User-Agent headers
User Controllable HTML Element Attribute (Potential XSS)	POST /login has user-controlled HTML attributes

5. Detailed Manual Testing Findings

Finding 1, NoSQL Injection

Field	Details
Severity	CRITICAL
OWASP Category	A03:2021, Injection
Affected URL	http://localhost:4000/allocations/1
Affected Parameter	User input field on the allocations page
HTTP Method	POST
CWE Reference	CWE-943: Improper Neutralization of Special Elements in Data Query Logic

5.1.1 Description

The /allocations endpoint passes user input directly into a MongoDB query without sanitization. Malformed input causes an unhandled MongoError crash, confirming NoSQL injection vulnerability. An attacker could manipulate query logic to bypass authorization or access other users' financial data.

5.1.2 Test Steps & Evidence

31. Logged into NodeGoat and navigated to http://localhost:4000/allocations/1
32. Entered payload: into the input field
33. Submitted the form
34. Result: MongoError: SyntaxError: unexpected token: identifier, app crashed



Oops..
MongoError: SyntaxError: unexpected token: identifier

Screenshot 8: NoSQL Injection on /allocations/1

Browser showing the MongoError crash page after submitting the payload on the allocations page

5.1.3 Impact

- Unauthorized access to other users' financial allocation data
- Authorization bypass via query operator manipulation
- Denial of service through repeated application crashes
- Full internal MongoDB error messages exposed to end users

5.1.4 Recommendations

35. Install validator library: `npm install validator`
36. Validate all inputs before passing to database queries
37. Use Mongoose with strict schemas for data type enforcement
38. Implement global error handlers never expose raw MongoDB errors

Finding 2, Injection / Code Parsing Error

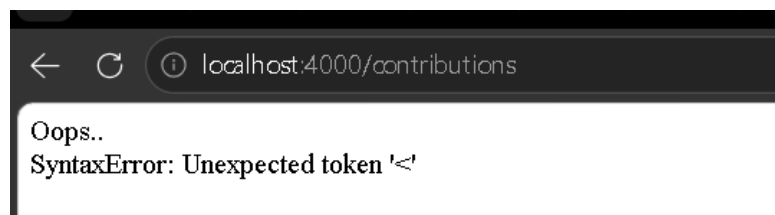
Field	Details
Severity	HIGH
OWASP Category	A03:2021, Injection
Affected URL	http://localhost:4000/contributions
Affected Parameter	Input fields on the contributions form
HTTP Method	POST
CWE Reference	CWE-20: Improper Input Validation

5.2.1 Description

The /contributions page attempts to evaluate user input as a structured expression. HTML tags submitted as input cause a JavaScript SyntaxError that is displayed directly to the user, confirming user input reaches an evaluation function without sanitization.

5.2.2 Test Steps & Evidence

39. Navigated to `http://localhost:4000/contributions`
40. Entered payload: `<img src=x onerror=alert('XSS')>` into input field
41. Result: Oops.. SyntaxError: Unexpected token '<' displayed on screen



Screenshot 9: Injection Error on /contributions

Browser showing the SyntaxError message on the contributions page after submitting the payload

5.2.3 Impact

- Application crashes and leaks internal code structure to attackers
- Potential escalation to Remote Code Execution in severe configurations
- Confirms dynamic code evaluation is used, an extremely dangerous practice

5.2.4 Recommendations

42. Never pass user input to eval(), Function(), or any dynamic parser
43. Validate all form inputs with strict type and format checks
44. Implement global Express error handler
45. Return only generic 'Invalid input' messages to users

Finding 3, Open Redirect / JavaScript URI Injection

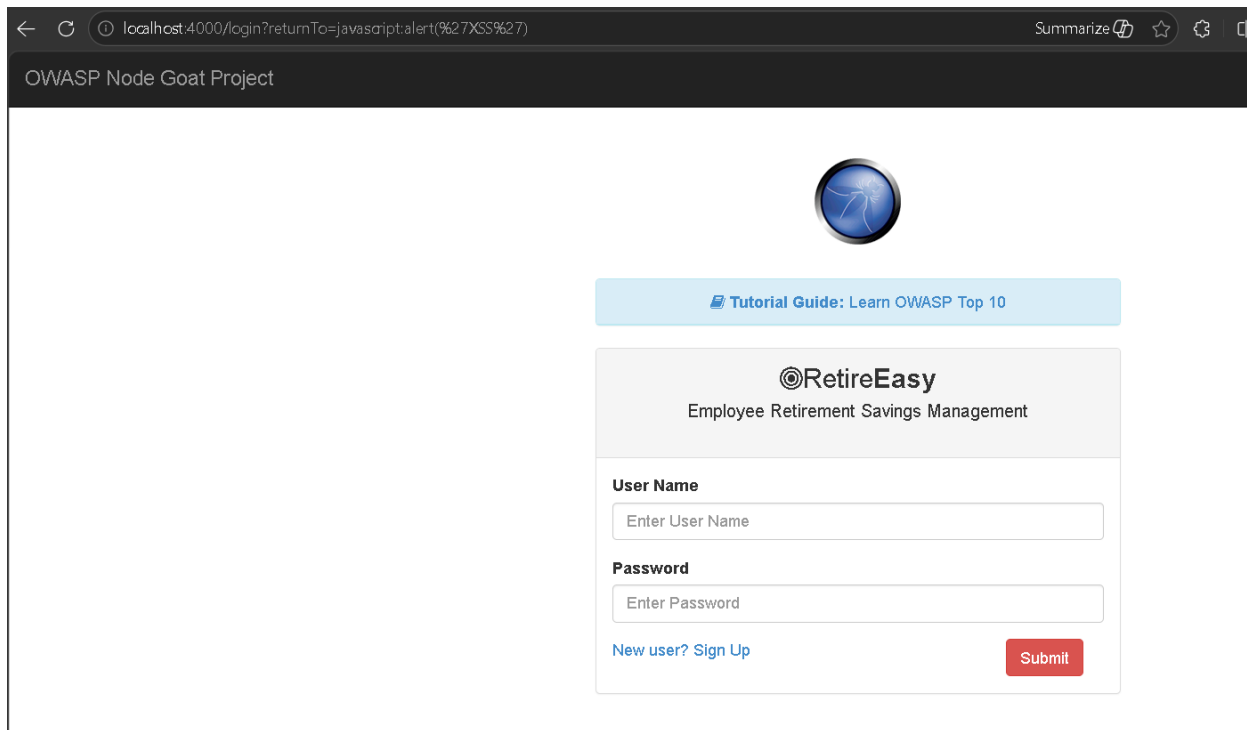
Field	Details
Severity	HIGH
OWASP Category	A01:2021, Broken Access Control
Affected URL	http://localhost:4000/login?returnTo=javascript:alert('XSS')
Affected Parameter	returnTo query parameter in the login URL
HTTP Method	GET
CWE Reference	CWE-601: URL Redirection to Untrusted Site (Open Redirect)

5.3.1 Description

The login page accepts an unvalidated returnTo parameter for post-login redirect. The application accepts javascript: URIs and external URLs, allowing attackers to craft malicious login links that redirect victims to phishing sites or execute JavaScript after authentication.

5.3.2 Test Steps & Evidence

46. Constructed URL: http://localhost:4000/login?returnTo=javascript:alert('XSS')
47. Entered valid credentials and submitted the login form
48. Result: Application accepted the javascript: URI as a valid redirect target



Screenshot 10: Open Redirect Test on Login Page

Browser showing the crafted returnTo URL and behavior observed after logging in

5.3.3 Impact

- Phishing attacks redirecting victims to fake login pages
- XSS execution via javascript: URI after authentication
- Session token theft if combined with other vulnerabilities

5.3.4 Recommendations

49. Only accept relative paths starting with / as valid returnTo values
50. Reject values containing http://, https://, javascript:, or //
51. Default to /dashboard if returnTo is absent or invalid

Finding 4, Weak Password Policy

Field	Details
Severity	CRITICAL
OWASP Category	A07:2021, Identification and Authentication Failures
Affected URL	http://localhost:4000/signup
Affected Parameter	Password field on the user registration form
HTTP Method	POST
CWE Reference	CWE-521: Weak Password Requirements

5.4.1 Description

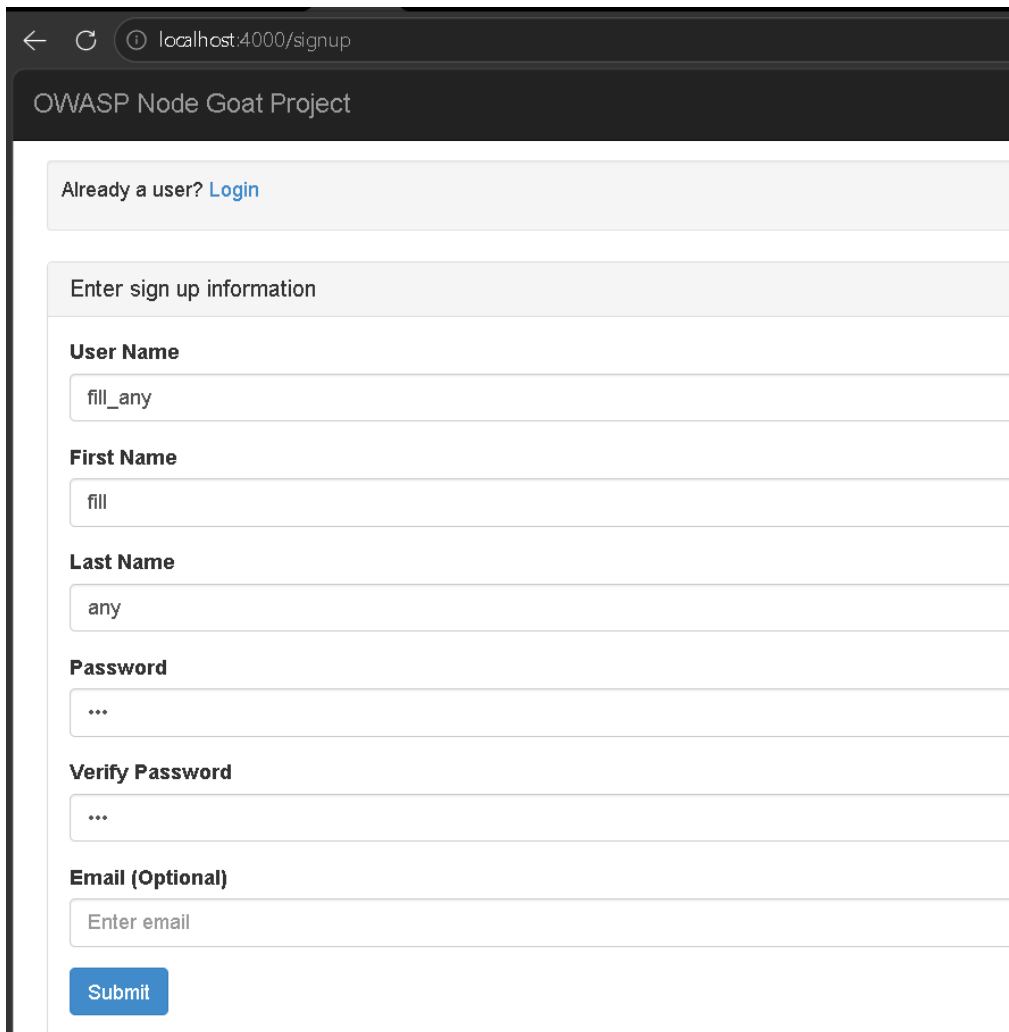
The registration form imposes no minimum password length or complexity requirements. An account was successfully created using the password '123' a 3-character all-numeric string crackable in milliseconds. The deprecated bcrypt-nodejs package is also used for hashing, which has known issues.

5.4.2 Test Steps & Evidence

52. Navigated to <http://localhost:4000/signup>

53. Entered: Username: fill_any, Password: 123 (3 characters, numeric only)

54. Result: Account created successfully, no validation error



← ↻ ⓘ localhost:4000/signup

OWASP Node Goat Project

Already a user? [Login](#)

Enter sign up information

User Name

fill_any

First Name

fill

Last Name

any

Password

...

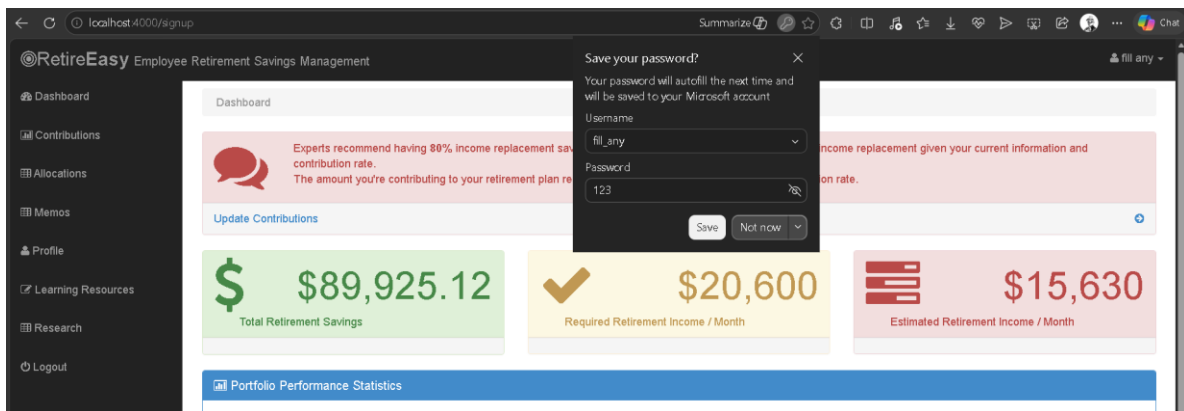
Verify Password

...

Email (Optional)

Enter email

Submit



Screenshot 11: Weak Password '123' Accepted at Signup

Browser showing successful account creation after registering with password '123'

5.4.3 Impact

- All user accounts vulnerable to brute-force and dictionary attacks
- Financial retirement data accessible to any attacker who cracks a weak password
- Non-compliance with NIST SP 800-63B, GDPR, PCI-DSS password requirements

5.4.4 Recommendations

55. Enforce minimum password length of 8+ characters
56. Require uppercase, lowercase, number, and special character
57. Replace bcrypt-nodejs with bcrypt: npm install bcrypt
58. Use salt rounds of at least 10: await bcrypt.hash(password, 10)
59. Implement account lockout after 5 failed login attempts

Finding 5, Verbose Error Message Disclosure

Field	Details
Severity	MEDIUM
OWASP Category	A05:2021, Security Misconfiguration
Affected URL	/allocations, /contributions, and other pages
Affected Parameter	All endpoints handling user input
HTTP Method	POST / GET
CWE Reference	CWE-209: Generation of Error Message Containing Sensitive Information

5.5.1 Description

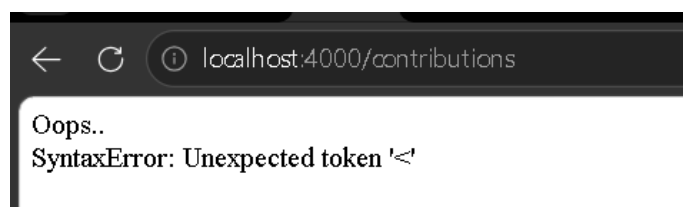
Raw internal error messages including full MongoDB error objects and JavaScript SyntaxError stack traces are displayed directly in the browser. This reveals the technology stack, database

type, query structure, and parsing mechanism, invaluable information for an attacker refining their payloads.

5.5.2 Evidence

Error 1 (/allocations): MongoError: SyntaxError: unexpected token: identifier

Error 2 (/contributions): Oops.. SyntaxError: Unexpected token '<'



Screenshot 12: Verbose Error Message Displayed to User

Browser showing raw MongoError or SyntaxError message rendered on the application page

5.5.3 Recommendations

60. Add global Express error handler: `app.use((err, req, res, next) => { logger.error(err); res.status(500).send('An error occurred.');` `});`
61. Set `NODE_ENV=production` to suppress stack traces
62. Install winston for server-side logging: `npm install winston`
63. Never send raw error details to the client

Finding 6, Incorrect Input Validation on Profile Page

Field	Details
Severity	LOW
OWASP Category	A03:2021, Injection
Affected URL	http://localhost:4000/profile
Affected Parameter	First Name and other profile fields
HTTP Method	POST
CWE Reference	CWE-20: Improper Input Validation

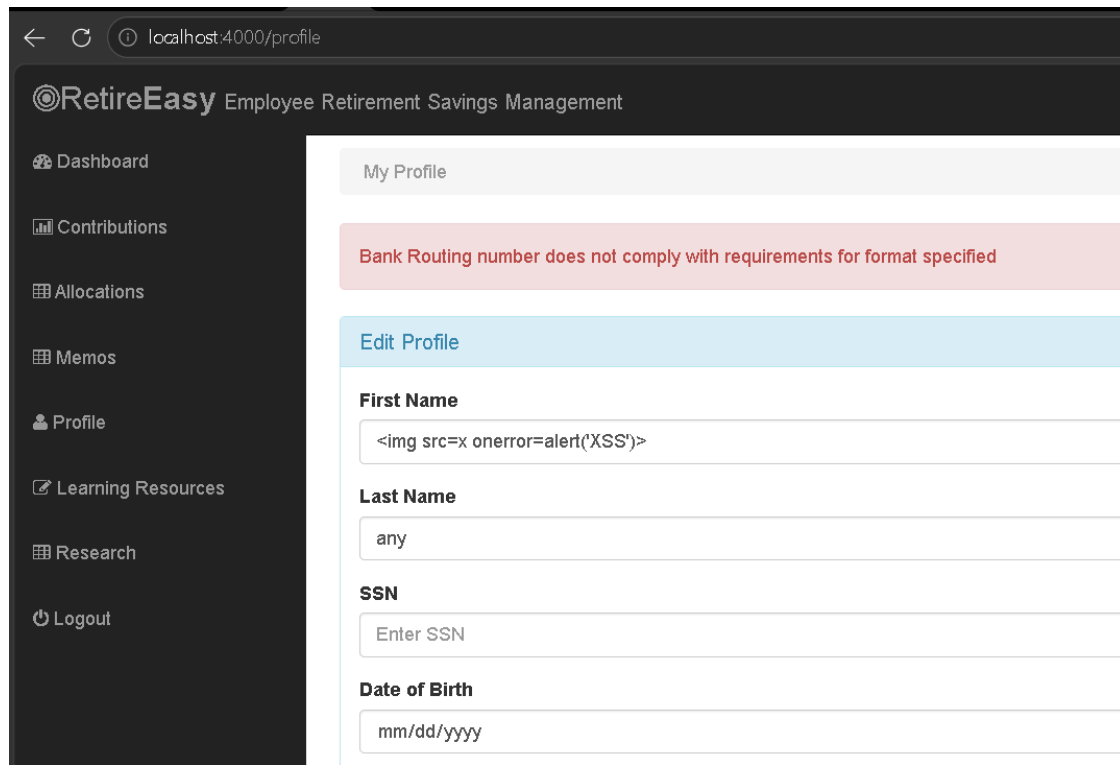
5.6.1 Description

XSS payload submitted in the First Name field returned the error: 'Bank Routing number does not comply with requirements for format specified.' This is clearly incorrect, the wrong validation rule is applied to the wrong field. While the XSS was blocked, this indicates validation logic is misconfigured across the application, potentially leaving other fields unprotected.

5.6.2 Evidence

Input: `<script>alert('XSS')</script>` entered in First Name field

Result: Bank Routing number does not comply with requirements for format specified



Screenshot 13: Incorrect Error Message on /profile

Browser showing the 'Bank Routing number' error after entering XSS payload in the First Name field

5.6.3 Recommendations

64. Audit all form validation rules, ensure each field has the correct validator
65. Use `validator.isAlpha()` for name fields, `validator.isEmail()` for email, etc.
66. Write unit tests for each form field to verify correct validation behavior

6. Risk Matrix

Finding	Likelihood	Impact	Overall Risk	Fix Priority
NoSQL Injection (/allocations)	High	Critical	Critical	Immediate (Week 2)
Weak Password Policy (/signup)	High	Critical	Critical	Immediate (Week 2)
XSS, DOM Based (/login)	High	High	High	Week 2
XSS, Persistent (/memos)	High	High	High	Week 2
SQL Injection (/contributions)	High	High	High	Week 2
Path Traversal (/allocations)	Medium	High	High	Week 2
Open Redirect (/login?returnTo=)	Medium	High	High	Week 2
Missing CSP Headers	Medium	Medium	Medium	Week 2 (Helmet.js)
Verbose Error Messages	Low	Medium	Medium	Week 2
Missing Security Headers (all)	Low	Low	Low	Week 2 (Helmet.js)
Incorrect Validation (/profile)	Low	Low	Low	Week 3

7. Areas of Improvement

7.1 Input Validation & Sanitization

- Install validator npm library and apply field-specific rules to all inputs
- Sanitize all inputs before they reach database queries or parsers
- Reject inputs containing HTML tags, script content, or query operators
- Apply validation on both client and server side

7.2 Password Security

- Enforce minimum 8-character password with uppercase, lowercase, number, special character
- Replace deprecated bcrypt-nodejs with actively maintained bcrypt package
- Hash passwords with salt factor of at least 10 before storing
- Implement account lockout after 5 failed attempts

7.3 Authentication & Session Management

- Replace session-cookie auth with JWT token-based authentication
- Validate and whitelist all redirect URLs, reject javascript: and external URIs
- Set secure and HttpOnly flags on all session cookies
- Add SameSite=Strict attribute to all cookies

7.4 Security Headers (via Helmet.js)

- Install Helmet.js: `npm install helmet`, apply `app.use(helmet())`
- Configure Content Security Policy (CSP) to prevent XSS
- Enable X-Frame-Options to prevent clickjacking
- Set X-Content-Type-Options, HSTS, and X-XSS-Protection headers

7.5 Error Handling & Logging

- Implement global Express error handler middleware
- Install winston: `npm install winston` for structured logging
- Return only generic error messages to users
- Log all errors server-side with timestamps and request context

7.6 General Security Hygiene

- Enable HTTPS for all traffic, never transmit credentials over plain HTTP
- Rate limit /login and /signup to prevent brute-force attacks
- Update all outdated JavaScript libraries (Bootstrap, jQuery)
- Disable directory browsing on all server paths
- Remove X-Powered-By header to avoid technology fingerprinting
- Store all secrets in environment variables, never hardcode in config files

8. Week 2 Remediation Plan

The following security fixes will be implemented in Week 2 to address the vulnerabilities identified in this report:

Task	Library / Command	Addresses
Sanitize & validate all user inputs	npm install validator	Findings 1,2,5,6 + ZAP XSS alerts
Hash passwords securely	npm install bcrypt	Finding 4
Implement JWT authentication	npm install jsonwebtoken	Finding 3
Add secure HTTP security headers	npm install helmet	ZAP Medium alerts (CSP, Clickjacking, X-Powered-By)
Set up server-side logging	npm install winston	Finding 5
Implement global error handler	Express middleware (no install needed)	Finding 5 + ZAP Error Disclosure
Validate & whitelist redirect URLs	Custom middleware	Finding 3 + ZAP Redirect alerts
Set secure cookie attributes	Express session config update	ZAP Cookie alerts

9. Conclusion

The Week 1 Security Assessment of the OWASP NodeGoat application successfully identified 33 total vulnerabilities, 6 through manual testing and 27 through the OWASP ZAP automated scan, including 2 Critical, 9 High, 6 Medium, 8 Low, and 8 Informational severity findings.

The two most critical manual findings, NoSQL Injection on /allocations and the complete absence of any password policy, represent risks that would allow an attacker to access and manipulate all users' financial retirement data in a real production environment. The ZAP scan additionally confirmed persistent and reflected XSS vulnerabilities, SQL injection on /contributions, path traversal on /allocations, and a complete absence of security headers across the entire application.

The setup process revealed important real-world challenges including MongoDB version compatibility, PATH configuration, Java requirements for security tools, and database initialization. These practical troubleshooting experiences are valuable learning outcomes of the Week 1 phase.

All critical and high severity findings will be remediated in Week 2 by implementing input validation (validator.js), bcrypt password hashing, JWT authentication, and Helmet.js security headers. Week 3 will conduct penetration testing using Nmap and verify the effectiveness of all applied fixes through re-scanning.

End of Week 1 Security Assessment Report